

The latter two permissions are there only to allow you a finer-grain configuration of permissions against a sequence.

A general synopsis of the GRANT and REVOKE commands is as follows:

```
GRANT <permission> ON SEQUENCE <sequence> TO <role>;
REVOKE <permission> ON SEQUENCE <sequence> FROM <role>;
```

The special keyword ALL encapsulates all the permissions applicable to a sequence.

In order to understand how privileges work for a sequence, let's consider the sequence used to generate the primary keys of the categories table: categories_pk_seq.

First of all, remove all privileges from the luca role so that he can no longer interact with the sequence:

```
forumbd=> REVOKE ALL
           ON SEQUENCE categories_pk_seq
           FROM luca;
REVOKE
```

Now, if the luca role tries to get a new value from the sequence, he gets a permission denied error:

```
forumbd=> SELECT nextval( 'categories_pk_seq' );
ERROR:  permission denied for sequence categories_pk_seq
```

Giving the sequence the USAGE privilege allows the luca role to query the sequence again:

```
forumbd=> GRANT USAGE ON SEQUENCE categories_pk_seq TO luca;
GRANT
```

Now, the role can successfully apply the setval function:

```
forumbd=> SELECT setval( 'categories_pk_seq', 10 );
setval
-----
      10
(1 row)

forumbd=> SELECT nextval( 'categories_pk_seq' );
nextval
-----
```

```
11
(1 row)
```

Remember that the `USAGE` privilege encapsulates both `SELECT` and `UPDATE` privileges, so once you have granted `USAGE` to a role, the sequence can be queried and set to a specific value.

Permissions related to schemas

A schema is a namespace for various objects, mainly tables and views, but also functions, routines, and other database objects. There are primarily two permissions that can be applied to a schema: `CREATE`, to allow the creation of objects within the schema, and `USAGE`, to allow the role to “use” objects in the schema (assuming it has appropriate permissions for the object).

That can look a little confusing at first since if the role does not have the `USAGE` permission, it will not be able to access the object even if it is the owner.

The general synopsis for using `GRANT` and `REVOKE` here involves the explicit `ON SCHEMA` clause (to distinguish them from permissions targeting a table):

```
GRANT <permission> ON SCHEMA <schema> TO <role>;
REVOKE <permission> ON SCHEMA <schema> FROM <role>;
```

As in other similar statements, the keyword `ALL` encapsulates all the permissions. In order to better understand the two different permissions, let’s create a configuration schema and see how to enable access to it:

```
-- as user forum
forumdb=> CREATE SCHEMA configuration;
CREATE SCHEMA
```

The schema was created by the `forum` user, and therefore the user `luca` does not have any privileges in it, so he is not able to create a table:

```
-- as user 'luca'
forumdb=> CREATE TABLE configuration.conf( param text,
                                             value text,
                                             UNIQUE (param) );

ERROR:  permission denied for schema configuration
LINE 1: CREATE TABLE configuration.conf( param text, value text, UNI...
```

In order to allow the user luca to create new objects within the schema, the CREATE permission has to be granted. However, without the USAGE permission, the role will not be able to access anything in the schema, so you need to provide both permissions at the same time:

```
-- as user 'forum'
forumdb=> GRANT CREATE ON SCHEMA configuration TO luca;
GRANT

forumdb=> GRANT USAGE ON SCHEMA configuration TO luca;
GRANT
```

Therefore, the luca role can now create a new object within the schema:

```
-- as user 'luca'
forumdb=> CREATE TABLE configuration.conf( param text,
                                           value text,
                                           UNIQUE (param) );

CREATE TABLE

forumdb=> INSERT INTO configuration.conf
        VALUES( 'posts_per_page', '10' );
INSERT 0 1
```

Without the USAGE permission, a role is no longer able to access any object within the schema, even if it is the owner of the object:

```
-- as role 'forum'
forumdb=> REVOKE USAGE ON SCHEMA configuration FROM luca;
REVOKE
```

In fact, a user can no longer read their own data:

```
-- as role 'luca'
forumdb=> SELECT * FROM configuration.conf;
ERROR:  permission denied for schema configuration
LINE 1: SELECT * FROM configuration.conf;
```

On the other hand, it is common to allow a role to manipulate data contained in a specific schema while not granting it the capability to create new database objects like tables.

This is a common scenario where a database administrator sets up a schema and its objects, leaving the final user to handle the data contained inside the schema, but not allowing them to modify the structure itself. This can be achieved with fine-grain permission setups, like:

```
-- as user 'forum'
forumdb=> GRANT USAGE ON SCHEMA configuration TO luca;
GRANT

forumdb=> REVOKE CREATE ON SCHEMA configuration FROM luca;
REVOKE
```

You can think of a schema as a container for other database objects. In order to access the container, you must have the `USAGE` permission, and in order to create new objects, you must have the `CREATE` permission. Nevertheless, `USAGE` does not provide you with unlimited access to any object within the schema. Instead, it provides you with access to objects depending on the permissions you have for such objects.

ALL objects in the schema

Since schemas are named containers of database objects, they can be used as a shortcut to apply different privileges to every object contained in the schema by means of the `ALL <objects> IN SCHEMA` clause.

By way of an example, in order to apply a set of equal permissions to all the tables contained in a schema, you can do the following:

```
-- as user 'forum'
forumdb=> REVOKE ALL
          ON ALL TABLES IN SCHEMA configuration
          FROM luca;
REVOKE

forumdb=> GRANT SELECT, INSERT, UPDATE
          ON ALL TABLES IN SCHEMA configuration
          TO luca;
GRANT
```

This can greatly simplify the management of large schemas.

At the moment, you can use the clause for the following:

- Tables, as in `ON ALL TABLES IN SCHEMA`
- Sequences, as in `ON ALL SEQUENCES IN SCHEMA`
- Routines, as in `ON ALL ROUTINES IN SCHEMA` (with the variants `ON ALL PROCEDURES IN SCHEMA` and `ON ALL FUNCTIONS IN SCHEMA`)

Permissions related to programming languages

Only a single permission applies to languages: `USAGE`. This permission allows a role to use the language. The special keyword `ALL`, which exists for compatibility with other `GRANT` and `REVOKE` statements, simply applies just that one permission.

It is a good security habit to grant as few permissions as possible in order to prevent untrusted users from running code within the database. As an example, to deny any role the ability to execute any snippet of PL/Perl code, you need to revoke the permission from the special group `PUBLIC`:



Note, this is only an example. If this language isn't installed on your system you may encounter an error.

```
forumdb=# REVOKE USAGE ON LANGUAGE plperl FROM PUBLIC;  
REVOKE
```

In this way, even a trusted user such as `luca` cannot execute a PL/Perl snippet:

```
forumdb=> DO LANGUAGE plperl $$ elog( INFO, "Hello World" ); $$;  
ERROR: permission denied for language plperl
```

If you want to allow the `luca` role exclusively to execute PL/Perl code, you need to grant this permission explicitly:

```
forumdb=# GRANT USAGE ON LANGUAGE plperl TO luca;  
GRANT
```

Permissions related to routines

The special keyword `ROUTINES` includes both `FUNCTIONS` and `PROCEDURES`. There is a single permission associated with `ROUTINES`, that is, the `EXECUTE` permission, in order to be able to run (execute) the code in the routine.

In order to demonstrate the permission, let's create a very simple routine, `get_max`, that returns the maximum between two integers:

```
forumdb=> CREATE FUNCTION get_max( a int, b int )
RETURNS int AS $$
BEGIN
  IF a > b THEN
    RETURN a;
  ELSE
    RETURN b;
  END IF;
END $$ LANGUAGE plpgsql;
```

Now, let's prevent any role apart from `luca` from executing such a routine:

```
forumdb=> REVOKE EXECUTE ON ROUTINE get_max FROM PUBLIC;
REVOKE
forumdb=> GRANT EXECUTE ON ROUTINE get_max TO luca;
GRANT
```

Any role other than `luca` will receive a `permission denied` error if invoking the function:

```
-- executing as enrico
forumdb=> SELECT forum.get_max( 10, 20 );
ERROR:  permission denied for function get_max
```

Since `get_max` is a function, we could have written the `GRANT` and `REVOKE` permission with the `FUNCTION` keyword instead of the catch-all `ROUTINE`. This is a matter of preference.

In particular, the `ROUTINE` keyword becomes handy when you want to apply permissions to all functions and procedures within a schema at the same time and with a single statement, something like the following:

```
- as user forum
forumdb=> GRANT EXECUTE ON ALL ROUTINES IN SCHEMA forum;
```

Permissions related to databases

There are a lot of permissions related to databases: `CONNECT` allows or rejects incoming connections without any regard to host-based access control; `TEMP` allows the creation of temporary objects (for example, tables) in a database; and `CREATE` allows the creation of new objects within a database.

The general synopsis is as follows:

```
GRANT <permission> ON DATABASE <database> TO <role>;  
REVOKE <permission> ON DATABASE <database> FROM <role>;
```

For instance, if you need to lock every user out of a database, for instance, because you have to do maintenance work, you can issue the following REVOKE command:

```
forumdb=# REVOKE CONNECT ON DATABASE forumdb FROM PUBLIC;  
REVOKE
```

New incoming connections will be rejected with a permission denied error:

```
$ psql -U luca forumdb  
psql: error: could not connect to server: FATAL: permission denied for  
database "forumdb"  
DETAIL: User does not have CONNECT privilege.
```

Now, if you want the luca role to be the only one able to connect to the database and create objects but not temporary ones, you need to issue the following command:

```
forumdb=# REVOKE ALL ON DATABASE forumdb FROM public;  
REVOKE  
  
forumdb=# GRANT CONNECT, CREATE ON DATABASE forumdb TO luca;  
GRANT
```

Other GRANT and REVOKE statements

There are other GRANT and REVOKE groups that control the permissions for table spaces, types, and foreign data wrappers. They will not be discussed here, but it is possible to find them in the official PostgreSQL documentation, and now that we have quite a clear workflow for applying permissions to different objects, they should be easy enough to understand.

Assigning the object owner

You have seen that the owner of an object has all the available permissions on such objects. Sometimes, you may wish to change the ownership of an object to another role, which, in turn, gets all the permissions. Usually, the change of ownership is done using a special ALTER statement such as the following:

```
ALTER <object> OWNER TO <role>;
```

For instance, to change the ownership of a table, you can issue the following command:

```
forumdb=# ALTER TABLE forum.categories OWNER TO luca;
ALTER TABLE
```

Whereas to change the ownership of a function, you can issue the following command:

```
-- equivalent to: ALTER FUNCTION get_max OWNER TO luca;
forumdb=# ALTER ROUTINE forum.get_max OWNER TO luca;
ALTER ROUTINE
```

Similar statements exist for all the other kinds of objects.

Superusers can alter the owner of every database object by setting it to any existing user, while normal users can change the ownership only to roles to which they belong.

Inspecting ACLs

In order to see which permissions have been granted to roles and objects, you can use the already mentioned `psql` special command `\dp` (describe permissions), which reports the ACLs configured for a specific object (a table, for instance). The command performs a query against the special catalog `pg_class`, which contains a specific field named `relacl` – an array of ACLs. You can see this as follows:

```
forumdb=> \dp users
```

Access privileges				
Schema	Name	Type	Access privileges	Column privileges
forum	users	table	forum=arwdDxt/forum	gecos: + forum_stats=w/forum

```
(1 row)
```



```
forumdb=> SELECT relname, relacl
           FROM pg_class WHERE relname = 'users';
 relname | relacl
-----+-----
 users   | {forum=arwdDxt/forum}
```

As you can see, the output from the `\dp` command and the query is the same, except for the formatting of the output.

You can also use the special function `aclexplode` to get more descriptive information about what the ACL means. The function returns a set of records, each one with the OID of the grantor and of the grantee and a textual description of the permission granted. It is therefore possible to build a query like the following:

```
forumdb=> WITH acl AS (
    SELECT relname,
           (aclexplode(relacl)).grantor,
           (aclexplode(relacl)).grantee,
           (aclexplode(relacl)).privilege_type
    FROM pg_class )
    SELECT g.rolname AS grantee,
           acl.privilege_type AS permission,
           gg.rolname AS grantor
    FROM acl
    JOIN pg_roles g ON g.oid = acl.grantee
    JOIN pg_roles gg ON gg.oid = acl.grantor
    WHERE acl.relname = 'users';
```

This returns all the individual permissions assigned to the table categories, as shown here:

grantee	permission	grantor
forum	INSERT	forum
forum	SELECT	forum
forum	UPDATE	forum
forum	DELETE	forum
forum	TRUNCATE	forum
forum	REFERENCES	forum
forum	TRIGGER	forum

RLS

In the previous part of the chapter, you saw the permission mechanism by which PostgreSQL allows roles (both users and groups) to access different objects within the database and the data contained in those objects.

In particular, with regard to tables, you learned how to restrict access to just a specific column list within the tabular data.

PostgreSQL provides another interesting mechanism to restrict access to tabular data: RLS. The idea is that RLS decides which tuples the role can have access to, either in read or write mode. Therefore, if the column-based permissions provide a way of limiting the vertical shape of the tabular data, RLS provides a way to restrict the horizontal shape of the data itself.

When is it appropriate to use RLS? Imagine you have a table that contains data related to users, and you don't want your users to be able to tamper with other users' data. In such a case, restricting the access of every user to just their own tuples provides good isolation that prevents the data from being tampered with. Another fairly common scenario is a multi-homed system, where you store the same data but for different companies in the very same tables. You don't want a company to be able to spy on or inspect the data of another company, so again RLS can prove useful.

Of course, RLS is not a silver bullet, and many of the solutions you could come up with involving RLS could have been realized with other techniques, but being aware of this important feature can make your data much more resistant to misuse.

The RLS infrastructure works on so-called policies. A policy is a set of rules according to which certain tuples should be made available to a user. Depending on the policies you apply, your roles (that is, users) will be able to read and/or write certain tuples.

Applying RLS to a table is usually a two-step process: first, you have to define a policy (or more than one), and then you have to enable the policy against the table. Please be aware that superusers, owners, and roles with the special `BYPASSRLS` property will not be subject to RLS.



ATTENTION: In the case of a database backup, for example, via `pg_dump`, the user who executes the backup must be able to bypass RLS policies; that is, it must have the `BYPASSRLS` property, or the backup will fail. Clearly, the superuser role (`postgres`) or any other role with the superuser option will succeed.

A policy defines the availability of tuples according to a logic criterion, that is, a filtering condition. A tuple can be available only for reading, only for writing, or for both. The general synopsis for a policy is as follows:

```
CREATE POLICY <name>  
ON <table>  
FOR <statement>
```